

A Technical Introduction to Generative adversarial network

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

Section 1

GANs often suffer from mode collapse where they fail to generalize properly, missing entire modes from the input data. For example, a GAN trained on the MNIST dataset containing many samples of each digit might only generate pictures of digit 0. This was termed "the Helvetica scenario". A typical mechanism for mode collapse is the generator only generating one or a few of the likely values, or a very incomplete picture of the target distribution. As the discriminator is only trained to distinguish real from fake samples, it will correctly identify the generated samples as real, but no penalty is imposed on the GAN's ability to generate data that represents the full range of the target distribution. Weak discriminators, for instance underparametrized ones, or ones trained too slow compared to the generator, may as well be unable to fully discriminate over the entire support of the distribution, and only become able to correctly discriminate a very incomplete part of the target distribution. Some researchers perceive the root problem to be a weak discriminative network that fails to notice the pattern of omission, while others assign blame to a bad choice of objective function. Many solutions have been proposed, but it is still an open problem. Even the state-of-the-art architecture, BigGAN (2019), could not avoid mode collapse. The authors resorted to "allowing collapse to occur at the later stages of training, by which time a model is sufficiently trained to achieve good results".

Section 2

The CycleGAN game is defined as follows: There are two probability spaces (Ω_X, μ_X) , (Ω_Y, μ_Y) , corresponding to the two domains needed for translations fore-and-back. There are 4 players in 2 teams: generators $G_X: \Omega_X \rightarrow \Omega_Y$, $G_Y: \Omega_Y \rightarrow \Omega_X$, and discriminators $D_X: \Omega_X \rightarrow [0, 1]$, $D_Y: \Omega_Y \rightarrow [0, 1]$. The objective function is $L(G_X, G_Y, D_X, D_Y) = L_{GAN}(G_X, D_X) + L_{GAN}(G_Y, D_Y) + \lambda L_{cycle}(G_X, G_Y)$.

Section 3

inpaint photographs generate fashion models, shadows, photorealistic renders of interior design, industrial design, shoes, etc. Such networks were reported to be used by Facebook. Some have worked with using GAN for artistic creativity, as "creative adversarial network". A GAN, trained on a set of 15,000 portraits from WikiArt from the 14th to the 19th century, created the 2018 painting Edmond de Belamy, which sold for US\$432,500. GANs were used by the video game modding community to up-scale low-resolution 2D textures in old video games by recreating them in 4k or higher resolutions via image training, and then down-sampling them to fit the game's native resolution (resembling supersampling anti-aliasing). In 2020, Artbreeder was used to create the main antagonist in the sequel to the psychological web horror series Ben Drowned. The author would later go on to praise GAN applications for their ability to help generate assets for independent artists who are short on budget and manpower. In May 2020, Nvidia researchers taught an AI system (termed "GameGAN") to recreate the game of Pac-Man simply by watching it being played. In August 2019, a large dataset consisting of 12,197 MIDI songs each with paired lyrics and melody alignment was created for neural melody generation from lyrics using conditional GAN-LSTM (refer to sources at GitHub AI Melody Generation from Lyrics).

Section 4

Mathematical The original GAN is defined as the following game: Each probability space $(\Omega, \mu_{\text{ref}})$ defines a GAN game. There are 2 players: generator and discriminator. The generator's strategy set is $\mathcal{P}(\Omega)$, the set of all probability measures μ_G on Ω . The discriminator's strategy set is the set of Markov kernels $\mu_D: \Omega \rightarrow \mathcal{P}[0, 1]$, where $\mathcal{P}[0, 1]$ is the set of probability measures on $[0, 1]$. The GAN game is a zero-sum game, with objective function $L(\mu_G, \mu_D) := \mathbb{E}_{x \sim \mu_{\text{ref}}, y \sim \mu_D(x)} [\ln y] + \mathbb{E}_{x \sim \mu_G, y \sim \mu_D(x)} [\ln(1 - y)]$.

Section 5

Generative reparametrization The GAN architecture has two main components. One is casting optimization into a game, of form $\min_G \max_D L(G, D)$, which is different from the usual kind of optimization, of form $\min_{\theta} L(\theta)$. The other is the decomposition of μ_G into $\mu_Z \circ G^{-1}$, which can be understood as a reparametrization trick. To see its significance, one must

compare GAN with previous methods for learning generative models, which were plagued with "intractable probabilistic computations that arise in maximum likelihood estimation and related strategies". At the same time, Kingma and Welling and Rezende et al. developed the same idea of reparametrization into a general stochastic backpropagation method. Among its first applications was the variational autoencoder.

Section 6

$L_G = -\mathbb{E}_{x \sim \mu_G} [D(x)]$ $\{\displaystyle L_G = -\operatorname{E}_{x \sim \mu_G} [D(x)]\}$ Least squares GAN: $L_D = \mathbb{E}_{x \sim \mu_{\text{ref}}} [(D(x) - b)^2] + \mathbb{E}_{x \sim \mu_G} [(D(x) - a)^2]$ $\{\displaystyle L_D = \operatorname{E}_{x \sim \mu_{\text{ref}}} [(D(x) - b)^2] + \operatorname{E}_{x \sim \mu_G} [(D(x) - a)^2]\}$